

MettaKernel: A MeTTa-hosted Proof/Inference Kernel for both Dependent Type Theory and Higher-Order Tarski–Grothendieck Set Theory

Design, Curriculum, and a Prototype-First Build Plan

Oruži

Zar Goertzel

2026-06-18 (draft)

Abstract

We set out to build one small *trusted* proof/inference kernel, written *in MeTTa* (as an ordinary MeTTa program, executed on the CeTTa engine), that can express and check proofs in *both* foundations we care about: CIC-grade *dependent type theory* (the foundation of Lean and Coq) and *higher-order Tarski–Grothendieck set theory* (HOTG, the classical higher-order logic of Megalodon). The kernel is a MeTTa program — not a new CeTTa language substrate — in the same spirit as `pverify`, an existing Metamath proof checker written in MeTTa. Two principles govern the work. First, **prototype a fully working kernel before any deeper integration into the engine**, so that the design is chosen empirically rather than frozen in advance. Second, **conversion is the danger line**: the kernel’s $\beta\eta\delta\iota$ conversion must be its own small, decidable, resource-bounded MeTTa functions and must never be delegated to CeTTa’s general runtime rewriting, or proof checking ceases to be a terminating judgment. We present the two reference kernels we mirror (the Lean `PureKernel` for DTT and Megalodon’s `tp/tm/pf+check_doc` core for HOTG), a graded *curriculum* of the easiest examples that pin down the smallest fully expressive kernel (a DTT ladder following Smolka & Brown’s *Introduction to Computational Logic*, and an HOTG ladder following Megalodon), and a prototype-first build plan whose first milestone is an honest *capacity ledger* (AGREE/DIVERGE/OPEN) rather than a claim of finality. The MeTTa kernel’s verdicts are treated as a *subject* to be measured against the oracles, not as an oracle itself.

1 Why a single kernel for two foundations

Lean/Coq rest on the Calculus of Inductive Constructions: dependent function and pair types, an inductive-definition mechanism with computation rules, a universe hierarchy, and a *decidable* definitional equality. Megalodon rests on HOTG: simple types with an impredicative `Prop`, proof terms, classical logic, set membership, extensionality, foundational ($\in$ -)induction, and a choice operator. These are different mathematics, but a *proof kernel* for either is small: it is a term/type/proof object language plus a conversion relation plus a checking judgment, with everything else (parsing, automation, optimization, the ambient runtime) living *outside* the trusted core.

Our thesis — developed at length in the companion memo *metta_proof_kernel_grounded* — is that MeTTa is an apt host for such a kernel *as a program*: terms, types, and proofs are MeTTa atoms; the checking and conversion judgments are MeTTa functions; theories and imports are MeTTa data. The trusted waist is then a small auditable MeTTa module, and the two foundations appear as two *checkers* sharing that waist. We are not proposing to encode both foundations into

one ambient rewriting theory (the Dedukti route); we keep the foundations’ judgments first-class and only share the machinery beneath them.

What this document is. A design + curriculum memo to drive a prototype-first build. It records: the reference kernels we mirror (§2); the MeTTa-program architecture (§3); the graded curriculum that defines “smallest fully expressive” (§4); the build plan and its first milestone, a capacity ledger (§5); the conversion discipline that is the central hazard (§6); and the open questions we deliberately leave open (§7).

2 The two reference kernels (our oracles and templates)

2.1 DTT: the Lean PureKernel

A trusted, *sorry-free* dependently-typed kernel already exists in Lean (Mettapedia/.../MeTTa/PureKernel/, 28 files). Its term language is

```
PureTm n ::= var (Fin n) | const DeclName | U0 | U1
          | pi A B | sigma A B | id A t u
          | lam B | app f a | pair t u | fst p | snd p | refl t
```

with two universes (U0 : U1), Π , Σ , and identity types. It provides one-step $\beta/\delta/\iota$ reduction, Church–Rosser confluence, HasType, a *decidable* definitional equality (defEqByNormalization?), subject reduction, and a bidirectional checker. Ordinary families (Bool, Nat, Unit) are *not* hard-coded: they enter through a general declaration mechanism (DeclSpec, InductiveDecl, RecursorDecl) with full well-formedness proofs. This is both our DTT *oracle* and our design template; crucially, the existing runtime-to-kernel bridge marks directExecEquivalence as false, so the runtime is *not* yet known to agree with this kernel.

2.2 HOTG: the Megalodon mathdata core

Megalodon’s trusted core (repos/megalodon-1.13/src/mathdata.mli) is genuinely compact — three small inductive types and a checker:

```
tp ::= TpVar i | Prop | Base i | Ar(tp, tp) | TpAll tp          (* simple types + type-vars
+ type-forall *)
tm ::= DB i | TmH h | Prim i | Ap(tm, tm) | Lam(tp, tm)
      | Imp(tm, tm) | All(tp, tm) | TTPAp(tm, tp) | TTPLam tm | TTPAll tm
pf ::= Gpa h | Hyp i | Known h | PTmAp(pf, tm) | PPfAp(pf, pf)
      | PLam(tm, pf) | TLam(tp, pf) | PTPAp(pf, tp) | PTPLam pf
```

plus check_doc (the proof checker), $\beta\eta\delta$ normalization with resource limits (BetaLimit, NonNormalTerm), and hash-addressed doc/theory/signa imports. Logic and set-theoretic content are *not* in this core: True/False/not/and/or are impredicative definitions, and membership/extensionality/choice/ $\in$ -induction enter as *axioms* in a preamble (Egal mode uses Eps choice and Tarski’s axiom; Mizar mode uses the and ZF-style axioms). The seed file Mettapedia/megalodon/experiments/01_basics.mg proves eight propositional theorems entirely by proof-term lambdas (e.g. andEL := fun A B H => H A (fun a b => a)).

2.3 Precedent that MeTTa can host a checker

hyperon/metamath/pverify (with `lib/pverify-utils.metta`) is a Metamath proof checker realized as a MeTTa program. It establishes feasibility of the *host*; it is Metamath, not HOTG, so it is precedent, not evidence about the present design.

3 Architecture: the kernel as a MeTTa program

Shared waist (written once in MeTTa). Atom encodings for binders (de Bruijn or locally nameless), α -safe substitution, typed contexts, declarations (parameter / definition / axiom / theorem / inductive / import), proof objects, the **conversion engine** ($\beta/\eta/\delta/\iota$ as explicit, terminating, resource-capped MeTTa functions), a **check/infer** driver, and hashing/provenance for imports. The two checkers below are *instances* of this waist — that sharing *is* the early unification.

HOTG checker. Mirror `tp/tm/pf + check_doc +  $\beta\eta\delta$` as MeTTa atoms and functions. Proof-term checking over impredicative **Prop**; classical/set/TG axioms are a *profile* layered above the core, admitted only after the bare proof-term checker is green.

DTT checker. Mirror **PureTm** (`U0/U1,  $\Pi, \Sigma, Id$`) with bidirectional `infer/check`, $\beta/\delta/\iota$, and decidable definitional equality by normalization; inductive families via **DeclSpec**, never hard-coded. Where the CeTTa engine’s `-profile he-prime` already provides dependent type inference, the MeTTa kernel may use it; new C features are added to **he-prime** *only* when the prototype proves a concrete need.

4 Curriculum: the examples that pin the smallest fully expressive kernel

The curriculum *defines* “fully expressive”: the smallest kernel is the one that checks every example below. The DTT ladder follows the chapter order of Smolka & Brown’s *Introduction to Computational Logic* (a Coq course by Megalodon’s own author), which introduces kernel features in increasing dependency order. The HOTG ladder follows Megalodon.

L0 — shared waist (both ladders ride on it). Names/binding, α -safe substitution, typed contexts, declarations, proof objects, conversion ($\beta/\eta/\delta/\iota$, decidable), hash imports + provenance, negative tests.

Table 1: DTT ladder (ICL spine + CeTTa **he-prime** seeds + **PureKernel**).

Step	Example (source)	Kernel feature it pins
D1	<code>bool/negb/andb/andb_com</code> (ICL §1.1)	inductive Bool , <code>match/recursor</code> , <code>def-eq</code> by computation
D2	<code>plus_0/plus_S/plus_assoc</code> (ICL §1.3–1.4)	Nat , <code>recursor</code> , structural induction
D3	<code>Π / proofs-as-programs</code> (ICL §1.2, 2.2, 2.6)	Π formation/intro/elim, application

Step	Example	Kernel feature
D4	Σ / pairs (ICL §6.6; he-prime TutPackHuman)	Σ formation/intro/projection
D5	Id/refl/cong/transport (ICL §3.3; he- prime TutEq/TutRefl/TutSuccCong)	identity types, J /transport
D6	app_assoc/length_app (ICL §8)	parametric inductive + recursor
D7	Vec/TutIndex safety; VecN -1 (open)	indexed inductive families; index-domain question (Number vs Nat)
D8	Even/ $\leq$ (ICL §7)	inductive predicates, elim restriction
D9	recursors + elim restriction + universes (ICL §5)	ι -reduction, proof irrelevance, universe consis- tency

Table 2: HOTG ladder (01_basics.mg + Megalodon +
mathdata tp/tm/pf).

Step	Example (source)	Kernel feature it pins
H1	propositional encodings (01_basics.mg, 8 thms)	tp(Prop), tm(All, Imp, Lam, Ap), pf (PLam, PPfAp, Hyp), β -normalization, check_doc
H2	$\forall$ -instantiation; $\rightarrow$ /proof application	PTmAp, PPfAp
H3	proof λ ; $\beta\eta\delta$; definition unfolding	δ on ThyDef/SignaDef, η
H4	equality/congruence (Leibniz eq)	polymorphic eq via TpAll
H5	membership/set/TG axioms (In/Empty/EmptyE, set-ext, In_ind, Eps)	<i>axiom profile</i> , admitted only after H1–H4 green

Cross-profile smoke. $P \rightarrow P$; $P \rightarrow Q \rightarrow P$; $P \wedge Q \rightarrow P$; $\perp \rightarrow P$; equality reflexivity/congruence; pair projections; and *one* tiny CeTTa operational-semantics theorem treated as a *certificate*, never as definitional equality.

Negative corpus (a fail-by-default gate — the soundness backstop). Bad de Bruijn index; unknown Known/Hyp; application type mismatch; a proposition-shape used as a proof object; cyclic/non-normal δ ; **an arbitrary MeTTa rewrite admitted as conversion** (the central danger); stale import/hash; VecN ...-1 wrongly accepted. The kernel must *reject* every item here; a checker that accepts any of them is unsound.

5 Prototype-first build plan

Phase 0. Shared MeTTa waist + the ledger harness (three deciders: MeTTa-kernel vs. Lean PureKernel oracle vs. he-prime C).

Phase 1 (HOTG prototype). MeTTa tp/tm/pf + proof-term checker + β ; *check the eight 01_basics.mg theorems green* (H1–H2).

Phase 2 (DTT prototype). MeTTa PureTm + bidirectional checker + $\beta/\delta/\iota$ + decidable def-eq; *D1–D5 green*; ledger vs. the Lean oracle and he-prime C.

Phase 3 (unify + harden). Both checkers share the waist; cross-profile smoke green; *negative corpus rejected*.

Phase 4 (climb the ladders). HOTG H3–H5 (definitions, equality, then set/TG axioms); DTT D6–D9 (lists, `Vec`, inductive predicates, recursors/universes); resolve the `VecN` index-domain question.

Later. Only after a fully working prototype and its lessons: consider deeper substrate integration — add the `he-prime` C features the prototype proved necessary; refine CeTTa. Not before.

Milestone 1 (the ledger, not a constitution). The deliverable is the working H1 + D1–D3 MeTTa kernel plus a populated *capacity ledger* with buckets AGREE-ACCEPT, AGREE-REJECT, DIVERGE, OPEN, plus the negative-corpus rejections. Divergences and open cases are first-class outputs, not blemishes to hide. Only then does external review become useful, and its question is whether the fragment boundary is honest and which divergences are bugs versus legitimate language semantics — not whether the final kernel is settled. The MeTTa-kernel verdicts are labelled **subject**; the Lean `PureKernel` and Megalodon are the **oracles**.

6 Conversion discipline (the central hazard)

The single biggest architectural risk is identifying kernel conversion with general MeTTa evaluation. We hold five lines: (A) decidable conversion $\neq$ general MeTTa eval — the kernel’s $\beta/\eta/\delta/\iota$ are its own bounded MeTTa functions (resource caps in the spirit of Megalodon’s `BetaLimit`); (B) MeTTa’s many-valued/nondeterministic typing vs. a functional DTT judgment — the kernel commits to a functional infer/check; (C) grounded/FFI values are a trusted boundary — grounded equality is not kernel reduction; (D) universe consistency is not optional forever, because proofs *about* MeTTa are a goal; (E) the kernel must be small and separable — a pure checking slice, not entangled in effectful evaluation.

7 Open questions (kept deliberately open)

(1) Should `VecN String -1` be accepted under `Number` indexing, or should tranche 1 restrict indexed families to `Nat`? (2) How much definitional equality belongs in the first tranche? (3) What stays in the trusted runtime core versus later self-hosted MeTTa rules? (4) What fragment of dependent typing is promised first? (5) Does the `PureKernel`-style core remain the convergence target for the runtime-facing tranche, given `directExecEquivalence = false`?

References (local anchors)

- G. Smolka and C. E. Brown, *Introduction to Computational Logic*, Lecture Notes, Saarland University, 2014. (DTT curriculum spine.)
- *metta_proof_kernel_grounded* (Aixiv, 2026): framework core, object-theory profiles, conversion discipline, hash imports.
- Megalodon: `repos/megalodon-1.13/src/mathdata.mli` (the HOTG `tp/tm/pf` + `check_doc` core); `examples/` (Egal/Mizar preambles); `experiments/01_basics.mg` (seed).

- `Lean PureKernel/README.md` and modules (the DTT oracle/template).
- `hyperon/metamath/pverify` (MeTTa-hosted checker precedent).